

Linux Task 02: Users, Groups & File Permissions

Part A: Understanding Users

```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ whoami
kali
kali@kali:~$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),107(bluetooth),124(lpadmin),132(wireshark),134(kabouter),135(vb
naxsf)
kali@kali:~$ cat /etc/passwd
root:x:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:11:11:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backups:x:34:34:backups:/var/backups:/usr/sbin/nologin
list:x:38:38:mailing list Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/usr/sbin/nologin
gnpt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-networkd:x:998:998:systemd Network Management:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd/bin/false
systemd-timesyncd:x:991:991:systemd Time Synchronization:/usr/sbin/nologin
messagebus:x:258:258:system message bus:/nonexistent:/usr/sbin/nologin
tss:x:101:101:TPM software stack:/var/lib/tpm/bin/false
stroumgwam:x:102:65534::/var/lib/stroumgwam:/usr/sbin/nologin
tcpdump:x:103:103::/nonexistent:/usr/sbin/nologin
smbd:x:989:65534:smbd user:/run/smbd:/usr/sbin/nologin
_nginx:x:104:65534::/run/nginx:/usr/sbin/nologin
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:105:105:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
n-opensmptd:x:106:106:NetworkManager OpenVPN:/var/lib/opensmptd:/usr/sbin/nologin
speech-dispatcher:x:107:29:Speech Dispatcher:/run/speech-dispatcher/bin/false
usbmux:x:108:40:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
n-opensmptd:x:109:110:NetworkManager OpenConnect plugin:/var/lib/NetworkManager/usbmux:/usr/sbin/nologin
pulse:x:110:111:PulseAudio daemon:/run/pulse:/usr/sbin/nologin
pipewire:x:988:988:system user for pipewire:/nonexistent:/usr/sbin/nologin
lightdm:x:111:111:Light Display Manager:/var/lib/lightdm:/bin/false
statd:x:112:65534::/var/lib/nfs:/usr/sbin/nologin
saned:x:113:113::/var/lib/saned:/usr/sbin/nologin
polkitd:x:987:987:User for polkitd:/usr/sbin/nologin
```

1. What is your current username?

My current username is the account I am currently logged into on the Linux system. It can be checked using the `whoami` command

2. What is UID?

UID (User ID) is a unique number assigned to every user in Linux. It helps the system identify users

3. What is GID?

GID (Group ID) is a unique number assigned to a group. It is used to manage permissions for users in the same group

4. What information does `/etc/passwd` contain?

The `/etc/passwd` file contains information about all user accounts on the system, including usernames, user IDs, group IDs, home directories, and login shells

Part B: Create Users & Groups

```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ sudo groupadd interns
[sudo] password for kali:
kali@kali:~$ sudo groupadd cyberteam
kali@kali:~$ sudo useradd student1
kali@kali:~$ sudo useradd student2
kali@kali:~$ sudo useradd student3
kali@kali:~$ groups student1
student1 : student1
kali@kali:~$ id student1
uid=1001(student1) gid=1003(student1) groups=1003(student1)
kali@kali:~$ id student2
uid=1002(student2) gid=1004(student2) groups=1004(student2)
kali@kali:~$
```

Part C: File Ownership

Item	Details
------	---------

Original Owner	kali
----------------	------

New Owner	student1
-----------	----------

Command used `sudo chown student1 report.txt`

```
(kali@kali)~[/cybersecurity_project]
$ ls
credentials.txt  notes.txt  report.txt
(kali@kali)~[/cybersecurity_project]
$ ls -l
total 0
-rw-rw-r-- 1 kali kali 0 Jun 14 13:14 credentials.txt
-rw-rw-r-- 1 kali kali 0 Jun 14 13:11 notes.txt
-rw-rw-r-- 1 kali kali 0 Jun 14 13:11 report.txt
(kali@kali)~[/cybersecurity_project]
$ sudo chown student1 report.txt
[sudo] password for kali:
(kali@kali)~[/cybersecurity_project]
$ ls -l
total 0
-rw-rw-r-- 1 kali kali 0 Jun 14 13:14 credentials.txt
-rw-rw-r-- 1 kali kali 0 Jun 14 13:11 notes.txt
-rw-rw-r-- 1 student1 kali 0 Jun 14 13:11 report.txt
```

Part D: File Permissions

```
(kali@kali)~[/]
$ touch security_policy.txt
(kali@kali)~[/]
$ ls
cybersecurity_lab  cybersecurity_project  Desktop  Documents  Downloads  Music  Pictures  Public  security_policy.txt  shell.php  spiderfoot-4.0  Templates  v4.0.tar.gz  Videos
(kali@kali)~[/]
$ ls -l security_policy.txt
-rw-rw-r-- 1 kali kali 0 Jun 14 14:08 security_policy.txt
(kali@kali)~[/]
$ chmod 444 security_policy.txt
(kali@kali)~[/]
$ ls -l security_policy.txt
-r--r--r-- 1 kali kali 0 Jun 14 14:08 security_policy.txt
(kali@kali)~[/]
$ chmod 664 security_policy.txt
(kali@kali)~[/]
$ ls -l security_policy.txt
-rw-rw-r-- 1 kali kali 0 Jun 14 14:08 security_policy.txt
(kali@kali)~[/]
$ chmod 777 security_policy.txt
(kali@kali)~[/]
$ ls -l security_policy.txt
-rwxrwxrwx 1 kali kali 0 Jun 14 14:08 security_policy.txt
```

Commands Used

`ls -l`

Purpose:

Displays detailed information about a file, including permissions, owner, group, size, and modification date

`chmod`

Purpose:

The `chmod` (Change Mode) command is used to modify file permissions and control who can read, write, or execute a file

Part E: Permission Analysis

1. Permission: 755 (rwxr-xr-x)

Owner Rights: The owner has read, write, and execute permissions. This means the owner can view the file, make changes to it, and run it if it is an executable file or script

Group Rights: Members of the group can read and execute the file but cannot modify its contents

Other User Rights: All other users can read and execute the file but are not allowed to make any changes

Explanation: Permission 755 is commonly used when a file or directory needs to be accessible to many users, but only the owner should be able to modify it. It provides a good balance between accessibility and security

Real-World Use Case: A website directory may use 755 permissions so that the web server can access and execute files while preventing unauthorized modifications

2. Permission: 644 (rw-r--r--)

Owner Rights: The owner can read and modify the file.

Group Rights: Group members can only read the file.

Other User Rights: Other users can also only read the file.

Explanation: This permission allows information to be shared with others while ensuring that only the owner can make changes. It is one of the most commonly used permission settings in Linux.

Real-World Use Case: Used for documents, reports, and configuration files that need to be viewed by multiple users but edited only by the owner.

3. Permission: 777 (rwxrwxrwx)

Owner Rights: The owner has full control over the file.

Group Rights: Group members also have full control.

Other User Rights : All users on the system have full control over the file.

Explanation : Permission 777 gives everyone permission to read, write, and execute the file. While it may seem convenient, it is considered very risky because anyone can modify or delete the file.

Real-World Use Case: It is sometimes used temporarily for testing purposes, but it should be avoided in production environments because it can lead to serious security issues.

4. Permission: 600 (rw-----)

Owner Rights :The owner can read and modify the file.

Group Rights: No permissions are granted to the group.

Other User Rights: No permissions are granted to other users.

Explanation: Permission 600 is used for highly sensitive files. Only the owner can access the file, making it one of the most secure permission settings.

Real-World Use Case: Used for password files, SSH private keys, confidential records, and personal information that should not be accessible to anyone else.

5. Permission: 700 (rwx-----)

Owner Rights: The owner has complete control over the file or directory, including reading, writing, and executing.

Group Rights: No permissions are granted.

Other User Rights: No permissions are granted.

Explanation: Permission 700 ensures that only the owner can access and use the file or directory. This helps protect sensitive data from unauthorized access.

Real-World Use Case: Commonly used for personal directories, administrative scripts, and private project folders that should only be accessed by the owner.

Part F: Security Challenge

File Name		Reason
password_backup.txt	600	Contains sensitive password information. Only the owner should be able to read and modify the file.
public_notice.txt	644	This file contains public information that everyone can read, but only the owner should be allowed to edit.
system_log.txt	640	System logs may contain important system details. The owner can read and modify them, while the group can only read them. Other users should not have access.
personal_notes.txt	600	Personal notes may contain private information and should only be accessible to the owner.

1. password_backup.txt Permission 600

Password backups contain highly sensitive information. Allowing other users to access this file could lead to security risks and unauthorized access to accounts. Therefore, only the owner should have permission to read and modify it.

2. public_notice.txt Permission 644

A public notice is meant to be shared with all users. Everyone should be able to read the information, but only the owner should be able to make changes to prevent unauthorized modifications.

3. system_log.txt Permission 640

System log files often contain details about system activities and errors. Administrators may need access to these logs, but regular users should not be able to view or modify them. This helps maintain system security and integrity.

4. personal_notes.txt Permission 600

Personal notes may contain private or confidential information. Restricting access to the owner helps protect personal data and prevents accidental exposure.

Part G: Linux Security Research

1. Why are file permissions important?

File permissions are important because they control who can access, modify, or execute files and directories on a system. They help protect sensitive information from unauthorized users and prevent accidental changes to important files. Proper permissions improve system security and ensure that users only have access to the resources they need.

2. What happens if sensitive files are given 777 permissions?

When a sensitive file is assigned 777 permissions, every user on the system gets full read, write, and execute access. This means anyone can view, modify, or even delete the file. Such permissions can lead to data leaks, unauthorized changes, malware infections, and serious security risks. Therefore, 777 permissions should be avoided for important or confidential files.

3. What is the Principle of Least Privilege?

The Principle of Least Privilege states that users should be given only the minimum permissions necessary to perform their tasks. They should not have access to files, systems, or resources that are not required for their work. This reduces the chances of accidental mistakes, misuse of privileges, and security breaches.

4. Why do organizations restrict user access?

Organizations restrict user access to protect sensitive data, maintain system security, and prevent unauthorized activities. Limiting access ensures that users can only view or modify information relevant to their roles. This helps reduce the risk of data theft, accidental errors, and cyberattacks while improving overall security management.

